

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем

Лабораторная работа №19

по дисциплине: Основы программирования

тема: **«Оценка сложности алгоритмов сортировки по времени»**

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич

Проверили:
ст. преп. Притчин Иван Сергеевич
асс. Черников Сергей Викторович

Белгород 2023 г.

Цель работы: получение навыков организации эксперимента и базового анализа результатов для оценки сложности алгоритмов по времени. Закрепление знаний о сортировках.

Содержание отчета:

- Тема лабораторной работы.
- Цель лабораторной работы.
- Результаты замеров (таблицы по заданию)
- Графики полученных зависимостей.
- Исходный код экспериментов.

Задание 2

Реализуйте алгоритм сортировки обменом (пузырьковую).

```
void bubbleSort(int *a, int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = size - 1; j > i; j--) {
            if (a[j - 1] > a[j]) {
                swap(&a[j - 1], &a[j]);
            }
        }
    }
}
```

Задание 3

Реализуйте алгоритм сортировки выбором.

```
void selectionSort(int *a, int size) {
    for (int i = 0; i < size - 1; i++) {
        int minPos = i;
        for (int j = i + 1; j < size; j++) {
            if (a[j] < a[minPos]) {
                minPos = j;
            }

            swap(&a[i], &a[minPos]);
        }
    }
}
```

Задание 4

Реализуйте алгоритм сортировки вставками.

```
void insertionSort(int *a, int size) {
    for (int i = 1; i < size; i++) {
        int t = a[i];
        int j = i;
        while (j > 0 && a[j - 1] > t) {
            a[j] = a[j - 1];
            j--;
        }

        a[j] = t;
    }
}
```

Задание 5

Реализуйте алгоритм сортировки расческой.

```
void combSort(int *a, int size) {
    int step = size;
    int swapped = 1;
    while (step > 1 || swapped) {
        if (step > 1) {
            step /= 1.24733;
        }
        swapped = 0;
        for (int i = 0, j = i + step; j < size; ++i, ++j)
            if (a[i] > a[j]) {
                swap(&a[i], &a[j]);
                swapped = 1;
            }
    }
}
```

Задание 6

Реализуйте алгоритм сортировки Шелла.

```
void shellSort(int *a, int size) {
    int j;
    for (int step = size / 2; step > 0; step /= 2)
        for (int i = step; i < size; i++) {
            int tmp = a[i];
            for (j = i; j >= step; j -= step) {
                if (tmp < a[j - step]) {
                    a[j] = a[j - step];
                }
                else {
                    break;
                }
            }
            a[j] = tmp;
        }
}
```

Задание 7

Реализуйте алгоритм цифровой сортировки.

```
int byteValue(int x, int i) {
    return (x >> (8 * i)) & BYTE_MASK;
}

void digitalSort(int *a, int size) {
    int bytes = sizeof(int);

    for (int byte = 0; byte < bytes; byte++) {
        int *output = (int *) malloc(size * sizeof(int));
        int count[256] = {0};

        for (int i = 0; i < size; i++) {
            count[byteValue(a[i], byte)]++;
        }

        for (int i = 1; i < 256; i++) {
            count[i] += count[i - 1];
        }

        for (int i = size - 1; i >= 0; i--) {
            output[count[byteValue(a[i], byte)] - 1] = a[i];
            count[byteValue(a[i], byte)]--;
        }

        for (int i = 0; i < size; i++) {
            a[i] = output[i];
        }

        free(output);
    }
}
```

Задание 8

Проведите замеры времени для всех реализованных алгоритмов с упорядоченными, неупорядоченными и упорядоченными в обратном направлении массивами размеров от 10000 до 100000 с шагом 10000.

#1 bubbleSort

A	B	C
random	ordered	orderedBackwards
Time	Time	Time
38.011 s.	13.380 s.	13.526 s.

#2 selectionSort

A	B	C
random	ordered	orderedBackwards
Time	Time	Time
Wrong!	32.337 s.	34.583 s.

#3 insertionSort

A	B	C
random	ordered	orderedBackwards
Time	Time	Time
7.683 s	0.000	0.000 s

#4 combSort

A	B	C
random	ordered	orderedBackwards
Time	Time	Time
0.022 s.	0.011 s.	0.010 s.

#5 shellSort

A	B	C
random	ordered	orderedBackwards
Time	Time	Time
0.029 s.	0.007 s.	0.007 s.

#6 digitalSort

A	B	C
random	ordered	orderedBackwards
Time	Time	Time
0.008 s.	0.010 s.	0.010 s.

Задание 9

Выполните модификацию проекта таким образом (посредством добавления новых функций, объявлением вспомогательных структур), чтобы можно было считать как и время, так и количество проводимых внутри операций сравнения, и так же проведите эксперимент с записью данных в файлы, только вместо времени выполнения записываете количество операций сравнения.

```
long long getBubbleSortNComp(int *a, int size) {
    long long nComps = 0;
    for (int i = 0; i < size - 1; i++) {
        for (int j = size - 1; j > i; j--) {
            if (++nComps && a[j - 1] > a[j]) {
                swap(&a[j - 1], &a[j]);
            }
        }
    }

    return nComps;
}
```

```
long long getSelectionSortNComp(int *a, int n) {
    long long nComps = 0;
    for (int i = 0; ++nComps && i < n; i++) {
        int min = a[i];
        int minIndex = i;
        for (int j = i + 1; ++nComps && j < n; j++)
            if (++nComps && a[j] < min) {
                min = a[j];
                minIndex = j;
            }

        if (++nComps && i != minIndex) {
            swap(&a[i], &a[minIndex]);
        }
    }

    return nComps;
}
```

```

long long getInsertionSortNComp(int *a, int size) {
    long long nComps = 0;
    for (int i = 1; i < size; i++) {
        int t = a[i];
        int j = i;
        while (j > 0 && a[j - 1] > t) {
            a[j] = a[j - 1];
            j--;
            nComps++;
        }

        a[j] = t;
    }

    return nComps;
}

```

```

long long getCombSortNComp(int *a, int size) {
    int step = size;
    int swapped = 1;
    long long nComps = 0;
    while (step > 1 || swapped) {
        if (step > 1) {
            step /= 1.24733;
        }
        swapped = 0;
        for (int i = 0, j = i + step; j < size; ++i, ++j)
            if (a[i] > a[j] && ++nComps) {
                swap(&a[i], &a[j]);
                swapped = 1;
            }
    }

    return nComps;
}

```

```

long long getShellSortNComp(int *a, int size) {
    int j;
    long long nComps = 0;
    for (int step = size / 2; step > 0; step /= 2) {
        for (int i = step; i < size; i++) {
            int tmp = a[i];
            for (j = i; j >= step; j -= step) {
                if (++nComps && tmp < a[j - step]) {
                    a[j] = a[j - step];
                } else {
                    break;
                }
            }

            a[j] = tmp;
        }
    }

    return nComps;
}

```


Задание 10

Проанализируйте результаты с помощью следующего инструмента, заполните таблицы, представленные на следующей странице.

Таблица 19.2: Затрачиваемое время сортировкой, чтобы упорядочить *N* элементов упорядоченного массива

Тип сортировки	10000	20000	30000	40000	50000	60000	70000	80000	90000	100000
Пузырьком	0.17	0.54	1.225	2.179	3.445	4.98	6.638	8.755	11.823	13.636
Выбором	0.311	1.366	1.319	2.665	3.754	5.501	9.664	11.621	11.958	19.697

Пузырьком => 1.38967710102803e-9*x^2
Выбором => 1.7793984137633e-9*x^2

Таблица 19.3: Затрачиваемое время сортировкой, чтобы упорядочить *N* элементов упорядоченного массива

Тип сортировки	50000 0	10000 00	15000 00	20000 00	25000 00	30000 00	35000 00	40000 00	45000 00	500000 0
Вставками	0.002	0.004	0.006	0.008	0.01	0.014	0.016	0. 016	0.017	0.02
Расческой	0.065	0.137	0.215	0.289	0.363	0.451	0.528	0.625	0.703	0.787
Шелла	0.038	0.077	0.122	0.235	0.236	0.328	0.317	0.364	0.417	0.502
Цифровая	0.046	0.077	0.116	0.166	0.198	0.33	0.325	0.35	0.405	0.488

Вставками => 9.36801799811921e-16*x^2
Расчёской => 1.5352727272657e-7*x
Шелла => 4.19541889056615e-10*x*log(x)^2
Цифровая => 9.2374025978021e-8*x

Таблица 19.4: Затрачиваемое время сортировкой, чтобы упорядочить N элементов массива упорядоченного в обратном порядке

Тип сортировки	10000	20000	30000	40000	50000	60000	70000	80000	90000	100000
Пузырьком	0.152	0.546	1.223	2.175	3.538	4.932	6.604	9.286	11.635	13.808
Выбором	0.188	0.605	1.401	2.361	3.793	5.491	10.08	14.427	15.775	14.374
Тип сортировки	50000	10000	15000	20000	25000	30000	35000	40000	45000	500000
	0	00	00	00	00	00	00	00	00	0
Вставками	0.002	0.004	0.006	0.007	0.01	0.012	0.014	0.015	0.019	0.019
Расческой	0.066	0.136	0.212	0.287	0.362	0.445	0.533	0.617	0.693	0.786
Шелла										
Цифровая	0.039	0.077	0.114	0.152	0.2	0.269	0.294	0.362	0.445	0.456

Пузырьком => $1.38967710102803e-9 \cdot x^2$
 Выбором => $1.76762207499141e-9 \cdot x^2$
 Вставками => $9.39643943005243e-16 \cdot x^2$
 Расчёской => $1.5255064934969e-7 \cdot x$
 Шелла => $4.05417092953514e-10 \cdot x \cdot \log(x)^2$
 Цифровая => $8.97454545421804e-8 \cdot x$

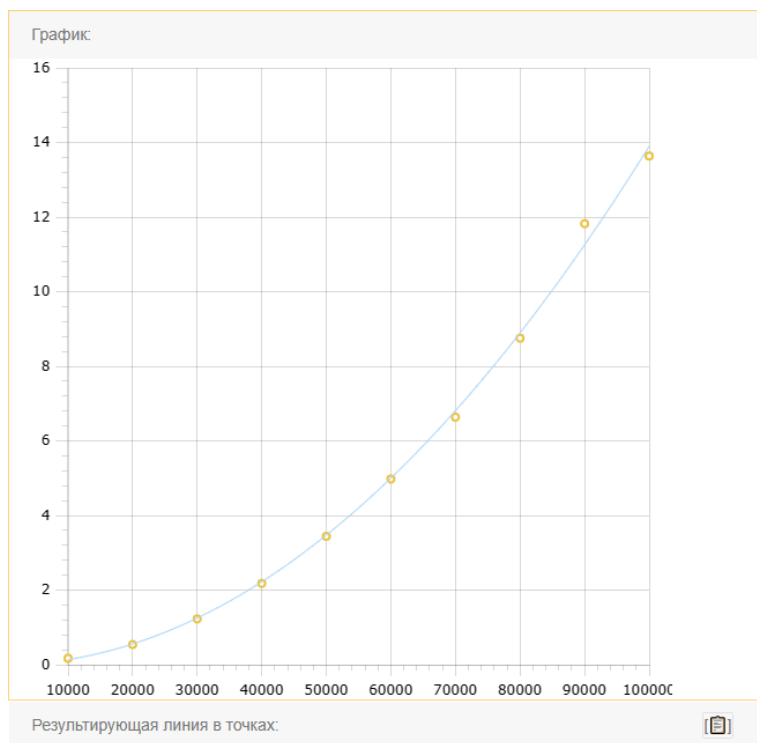
Таблица 19.5: Затрачиваемое время каждой из сортировок для того, чтобы упорядочить N элементов неупорядоченного массива

Тип сортировки	10000	20000	30000	40000	50000	60000	70000	80000	90000	100000
Пузырьком	0.391.	1.444	3.342	6.3	9.805	13.901	22.523	30.168	31.154	44.945
Выбором										
Тип сортировки	50000	10000	15000	20000	25000	30000	35000	40000	45000	500000
	0	00	00	00	00	00	00	00	00	0
Вставками										
Расческой	0.146	0.269	0.405	0.545	0.682	0.839	0.99	1.127	1.418	1.431
Шелла	0.263	0.378	0.592	1.381	1.187	1.395	1.737	2.089	2.235	2.534
Цифровая	0.036	0.071	0.117	0.144	0.181	0.331	0.491	0.347	0.449	0.402

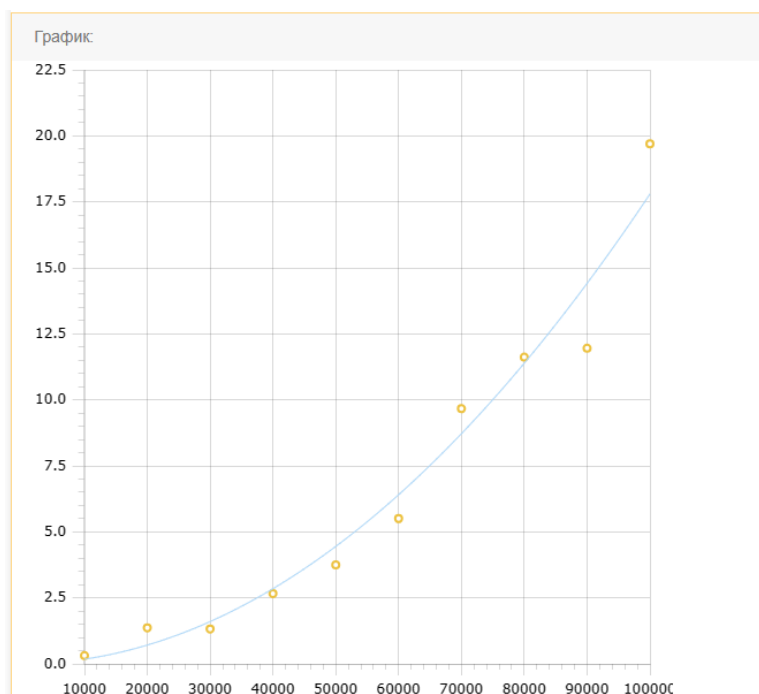
Пузырьком => $T(N) = 4.31648758526221e-9 \cdot x^2$
 Выбором => -
 Вставками => -
 Расчёской => $2.88524675323811e-7 \cdot x$
 Шелла => $2.19297623924086e-9 \cdot x \cdot \log(x)^2$
 Цифровая => $9.49090909205354e-8 \cdot x$

Графики 19.2 - 3: Затрачиваемое время сортировки, чтобы упорядочить N элементов упорядоченного массива

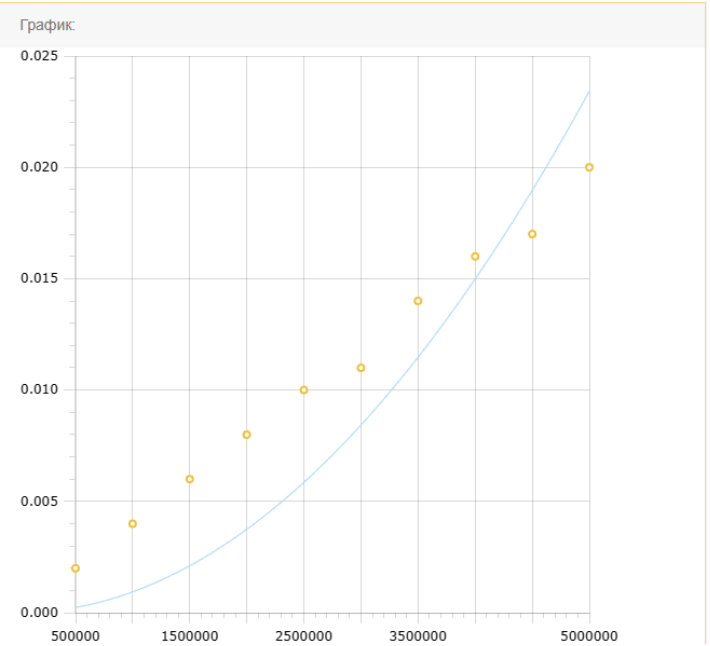
Пузырьком



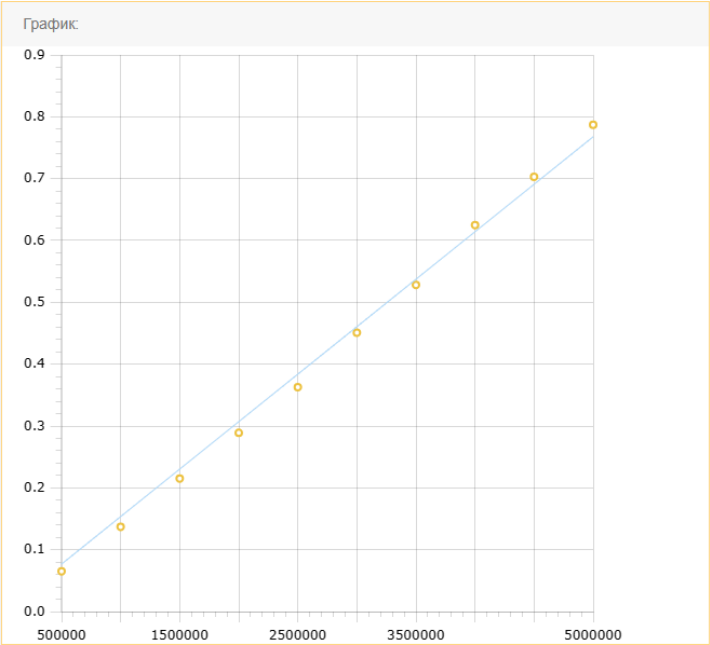
Выбором



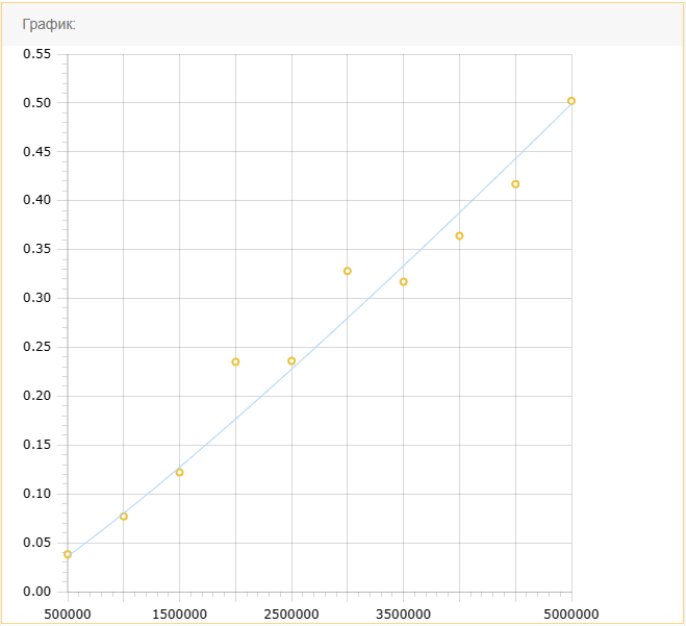
Вставками



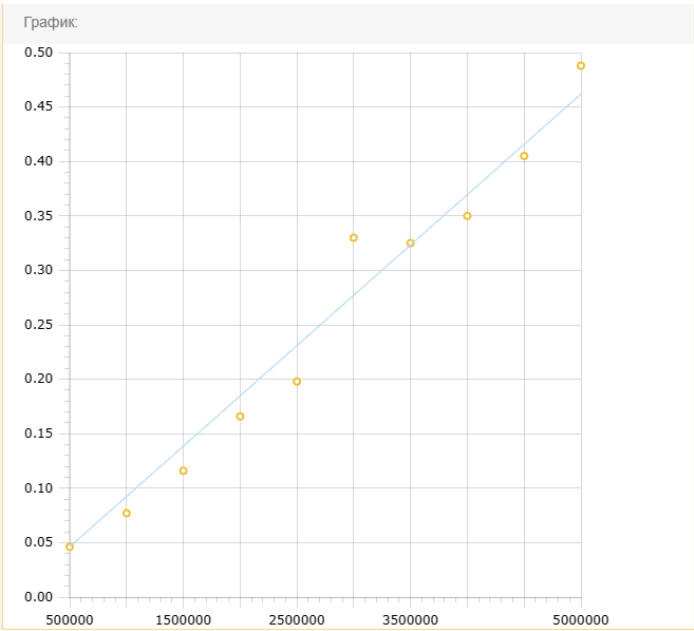
Расчёской



Шелла

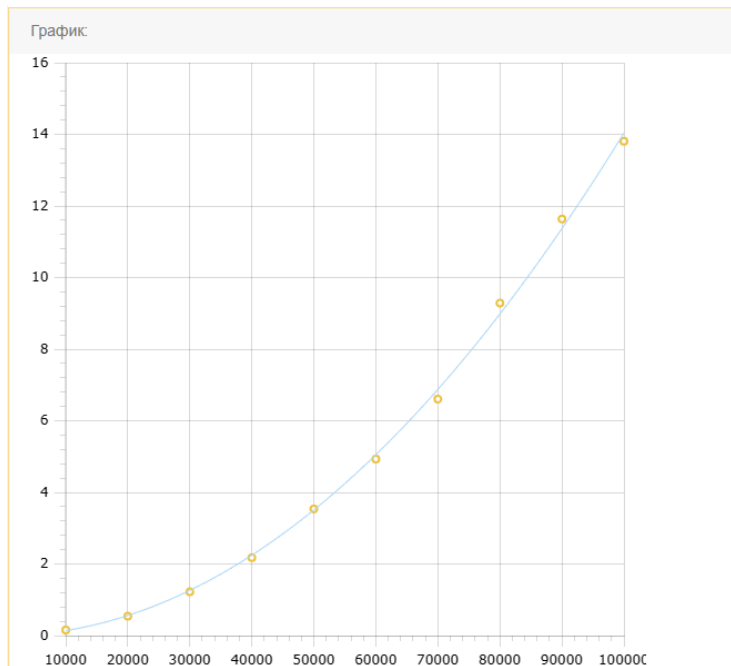


Цифровая

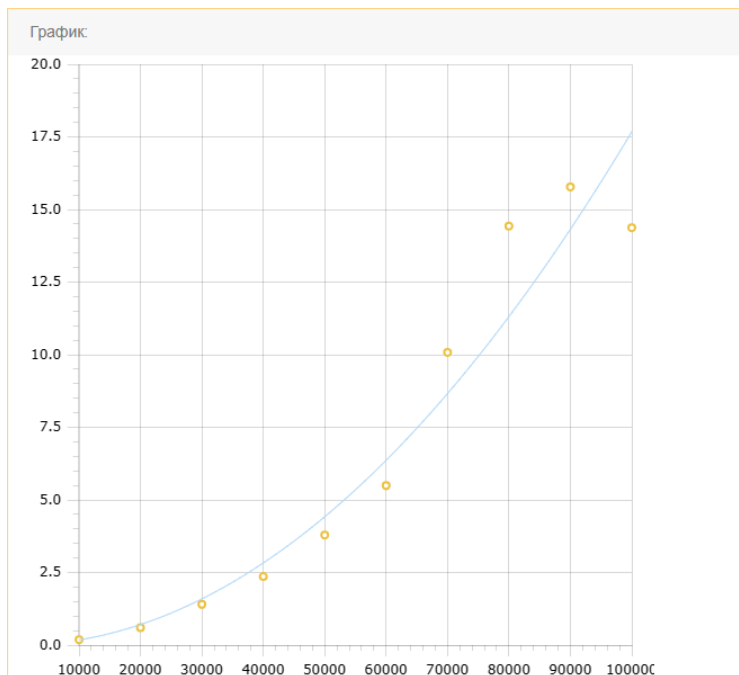


Графики 19.4: Затрачиваемое время сортировкой, чтобы упорядочить N элементов массива упорядоченного в обратном порядке

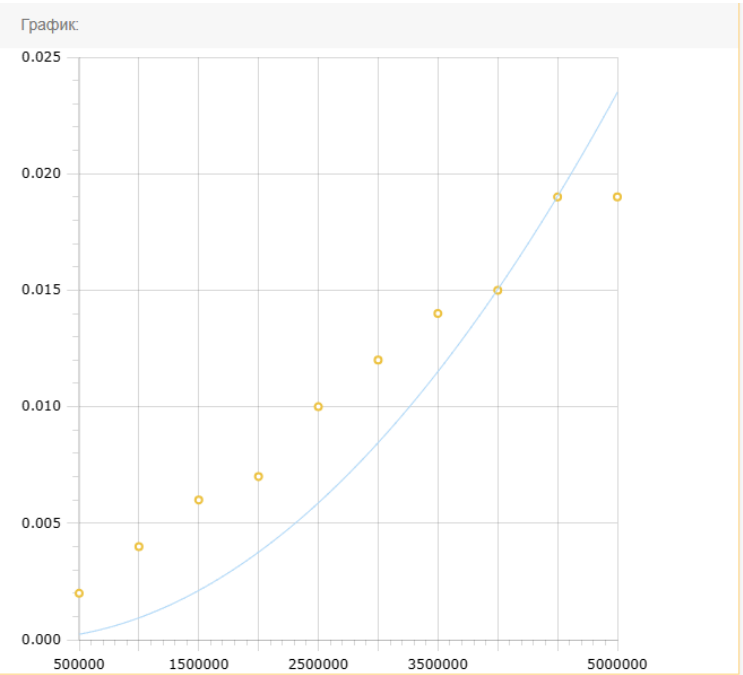
Пузырьком



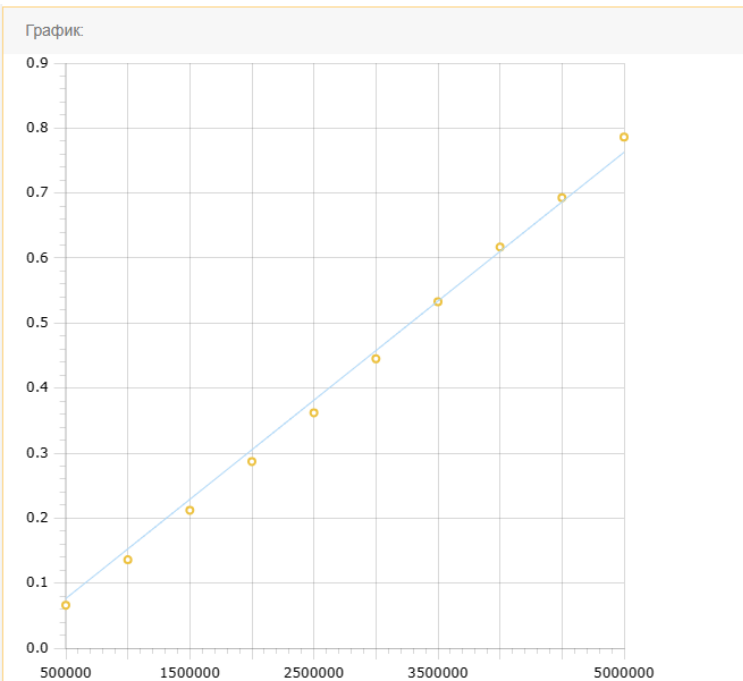
Выбором



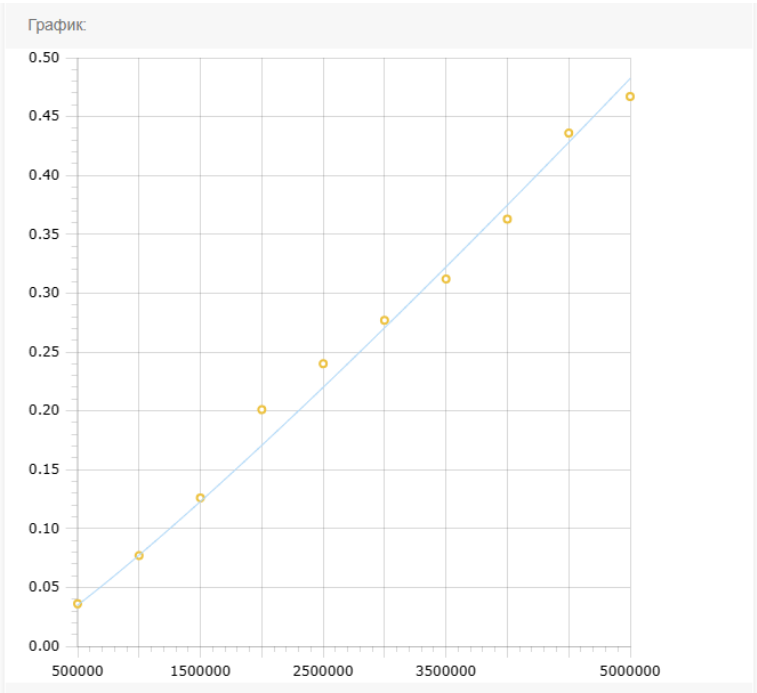
Вставками



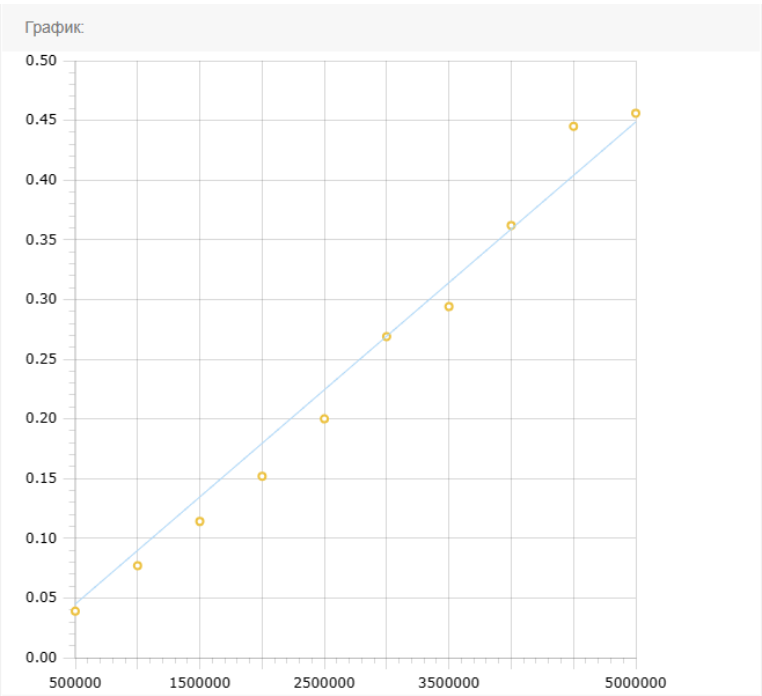
Расчёской



Шелла

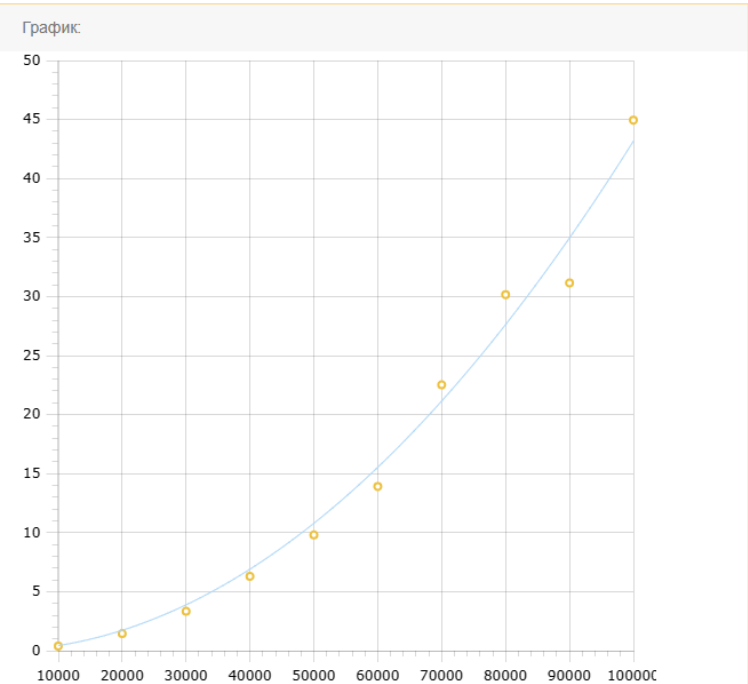


Цифровая



Графики 19.5: Затрачиваемое время каждой из сортировок для того, чтобы упорядочить N элементов неупорядоченного массива

Пузырьком



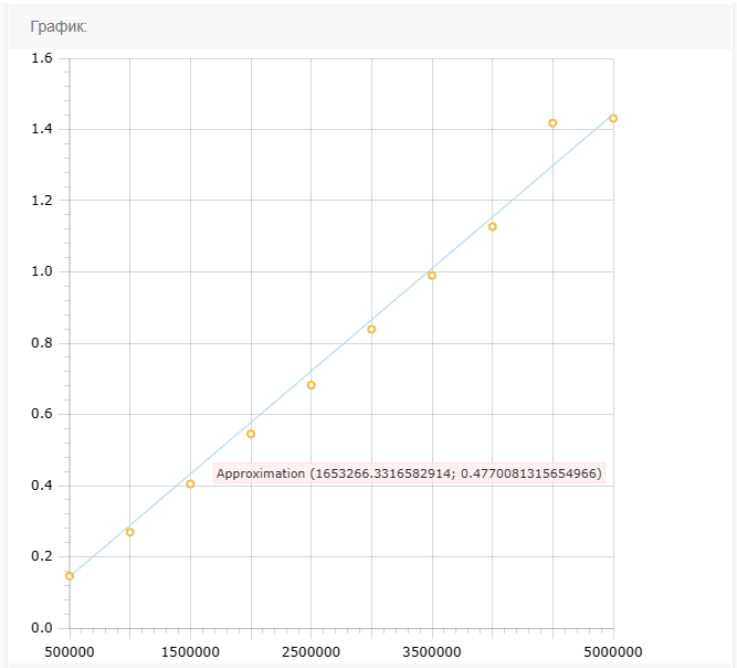
Выбором

Wrong!

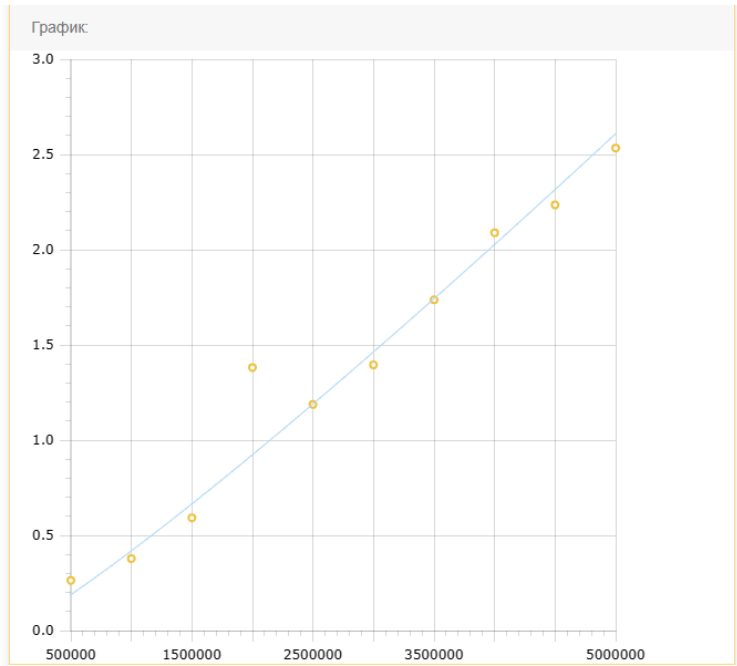
Вставками

-

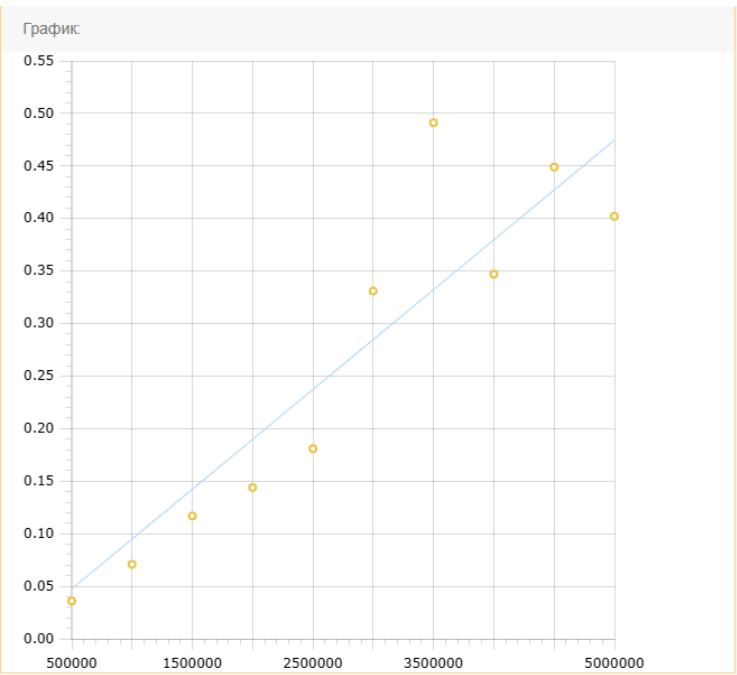
Расчёсой



Шелла



Цифровая



Дайте развернутый ответ на следующие вопросы:

1. При измерении времени выполнения, конечно же, влияет не только количество сравнений, но и операции обмена. Если бы вам было необходимо сортировать обменной сортировкой массив из структур большого размера (скажем, 8096 байт) – как можно было бы избежать «просадок» в скорости сортировки, не меняя алгоритм сортировки? Помните, что нам важно получить правильный порядок элементов, но для этого они сами не обязаны располагаться в правильном порядке.

1) Сортировка указателей. Аналогично сортировке индексов, можно создать массив указателей на структуры и сортировать его вместо сортировки самого массива структур. Это также снизит нагрузку на обмен данными и ускорит процесс сортировки.

2) Сортировка по ключам. Если структуры содержат определенный ключ, который используется для сравнения, можно создать массив этих ключей, а затем сортировать только массив ключей. После сортировки ключей, можно обновить позиции структур в исходном массиве в соответствии с отсортированными ключами.

2. Возьмите произвольную сортировку. Оцените, сколько времени потребуется ей для упорядочения массива из N ($N \geq 200000$) элементов. Все вычисления приложите к отчёту. Выполните запуск выбранной сортировки на вашей ЭВМ.

Оцените погрешность измерений (укажите отклонение в процентах).

Выбранная сортировка: выбором.

$T(200000) = 3.447 \cdot 10^{-10} \cdot 200000^2 = 13.788$ секунд

Size: 200000

Run #1| Name: selectionSort_random_time

Status: OK! Time: 13.893 s.

Отклонение на 0,76%.

3. * Если бы мы использовали сортировку вставками для сортировки двусвязного линейного списка, а не массива, то от какого недостатка этого алгоритма мы бы избавились, по сравнению с сортировкой массива?

4. * Подумайте, можно ли применить вашу реализацию цифровой сортировки к дробным числам? Что нужно изменить, чтобы она работала? Не обязательно реализовывать что-то, главное - описать идею. Ответить на эти вопросы поможет стандарт IEEE-754.